```
import numpy as np
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt
```

```
iris = load_iris()
X = iris.data
y = iris.target
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

```
class NaiveBayesClassifier:
    def fit(self, X, y):
        self.classes = np.unique(y)
        self.mean = {}
        self.var = {}
        self.priors = {}
        for c in self.classes:
            X_c = X[y == c]
            self.mean[c] = X_c.mean(axis=0)
            self.var[c] = X_c.var(axis=0)
            self.priors[c] = X_c.shape[0] / X.shape[0]

    def predict(self, X):
        y_pred = [self._predict(x) for x in X]
        return np.array(y_pred)

    def _predict(self, x):
        posteriors = []
        for c in self.classes:
            prior = np.log(self.priors[c])
            likelihood = -0.5 * np.sum(np.log(2 * np.pi * self.var[c]))
            likelihood -= 0.5 * np.sum(((x - self.mean[c]) ** 2) / self.var[c])
            posterior = prior + likelihood
            posteriors.append(posterior)
        return self.classes[np.argmax(posteriors)]
```

```
custom_nb = NaiveBayesClassifier()
custom_nb.fit(X_train, y_train)
y_pred_custom = custom_nb.predict(X_test)
```

```
gnb = GaussianNB()
gnb.fit(X_train, y_train)
y_pred_builtin = gnb.predict(X_test)
```

```
metrics = pd.DataFrame({
    "Model": ["Custom Naive Bayes", "Built-in GaussianNB"],
    "Accuracy": [
        accuracy_score(y_test, y_pred_custom),
        accuracy_score(y_test, y_pred_builtin)
    ],
    "Precision": [
        precision_score(y_test, y_pred_custom, average='macro'),
        precision_score(y_test, y_pred_builtin, average='macro')
    ],
    "Recall": [
        recall_score(y_test, y_pred_custom, average='macro'),
        recall_score(y_test, y_pred_builtin, average='macro')
    ],
    "F1-Score": [
        f1_score(y_test, y_pred_custom, average='macro'),
        f1_score(y_test, y_pred_builtin, average='macro')
```

```
    ]
```

```
print("\nPerformance Comparison:\n")
print(metrics)
```

```
Performance Comparison:

                Model  Accuracy  Precision    Recall  F1-Score
0   Custom Naive Bayes  0.977778    0.97619  0.974359  0.974321
1  Built-in GaussianNB  0.977778    0.97619  0.974359  0.974321
```

```
cm_custom = confusion_matrix(y_test, y_pred_custom)
cm_builtin = confusion_matrix(y_test, y_pred_builtin)
```

```
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
sns.heatmap(cm_custom, annot=True, cmap='Blues', fmt='d', ax=axes[0])
axes[0].set_title('Custom Naive Bayes')
axes[0].set_xlabel('Predicted')
axes[0].set_ylabel('Actual')

sns.heatmap(cm_builtin, annot=True, cmap='Greens', fmt='d', ax=axes[1])
axes[1].set_title('Built-in GaussianNB')
axes[1].set_xlabel('Predicted')
axes[1].set_ylabel('Actual')
plt.tight_layout()
plt.show()
```